Python Errors and Debugging

Decode error messages and fix bugs faster.

Python | Beginner | Ages 9–16

💡 The Big Idea

- An error message is Python telling you what went wrong and where.
- A bug is a mistake in code. Debugging means finding and fixing it.
- Every programmer, even the experts, sees errors all day. They are normal.
- Reading the message carefully usually shows the fix.

🔍 How to Read an Error

- Read the LAST line first. It names the error type and the reason.
- Then look at the line number the message points to.
- If it looks fine, check the line above. Missing brackets often show up on the next line.
- Change one thing at a time, then run again.

🧪 Debugging Toolbox

- Add `print()` lines to see what a variable holds.
- Run the program in small pieces.
- Explain your code out loud to a friend or a rubber duck.
- Predict what the code will do before running it.
- Take a short break. Fresh eyes spot mistakes fast.

🛡️ Handling Errors on Purpose

- `try:` runs code that might fail.
- `except ValueError:` runs if that specific error happens.
- Use it when you expect trouble, like a user typing text instead of a number.
- Catch specific errors, not everything.

💻 Errors You Will Meet

Reading a traceback python

```python
total = 10
count = 0
print(total / count)
```

▶ **Error message:**

```
Traceback (most recent call last):
  File "main.py", line 3, in <module>
    print(total / count)
ZeroDivisionError: division by zero
```

The last line names the error. The line above it shows where.

SyntaxError (missing colon) python

```python
for i in range(3)
    print(i)
```

▶ **Error message:**

```
SyntaxError: expected ':'
```

Fix: add a colon at the end of the for line.

IndentationError (missing indent) python

```python
for i in range(3):
print(i)
```

▶ **Error message:**

```
IndentationError: expected an indented block after 'for' statement on line 1
```

Fix: indent the lines inside the loop by 4 spaces.

NameError (typo in a variable) python

```python
score = 10
print(scor)
```

▶ **Error message:**

```
NameError: name 'scor' is not defined
```

Fix: spell the variable the same way every time.

TypeError (mixing text and numbers) python

```python
print("Age: " + 10)
```

▶ **Error message:**

```
TypeError: can only concatenate str (not "int") to str
```

Fix: print("Age: " + str(10)) or print("Age:", 10).

ValueError (wrong kind of value) python

```python
age = int("twelve")
```

▶ **Error message:**

```
ValueError: invalid literal for int() with base 10: 'twelve'
```

Fix: only convert text that looks like a number, such as int("12").

IndexError (position does not exist) python

```python
colors = ["red", "blue"]
print(colors[2])
```

▶ **Error message:**

```
IndexError: list index out of range
```

A list with 2 items only has indexes 0 and 1.

Handle an error with try / except python

```python
try:
    age = int("twelve")
except ValueError:
    print("Please type a number")
print("The program keeps going")
```

▶ **Output:**

```
Please type a number
The program keeps going
```

The program catches the error and carries on.

Debug with print python

```python
total = 0
for n in [3, 5, 7]:
    total = total + n
    print("n =", n, "total =", total)
print("Final:", total)
```

▶ **Output:**

```
n = 3 total = 3
n = 5 total = 8
n = 7 total = 15
Final: 15
```

The extra print line shows how total changes at every step.

📚 Key Terms

Bug
A mistake in code.

Debugging
Finding and fixing bugs.

Traceback
The message Python prints when an error happens.

SyntaxError
Python cannot understand the code. Often a missing colon or bracket.

IndentationError
The spaces at the start of a line are wrong.

NameError
A name was used that Python does not know.

TypeError
The wrong kind of value was used.

ValueError
The type is right but the value is not.

IndexError
A list position does not exist.

try / except
A way to catch an error and carry on.

🧮 Quick Reference

```
SyntaxError      -> check : ( ) " '
IndentationError -> check spaces
NameError        -> check spelling
TypeError        -> check str / int
ValueError       -> check the value
IndexError       -> check list length
KeyError         -> check dictionary key
ZeroDivisionError -> divisor is 0
```

✏️ Worked Examples

Q: Fix this line: print("Hi"
A: The closing bracket is missing, which causes a SyntaxError. Add it: print("Hi").

Q: A program says NameError: name 'nmae' is not defined.
A: Look for the typo. The variable should be name, not nmae.

Q: A calculator crashes when the user types abc instead of a number.
A: Wrap int(input()) in try / except ValueError and print a friendly message.

🚫 Common Mistakes

- Reading the first line of the traceback instead of the last.
- Changing many things at once, so you do not know what fixed it.
- Ignoring the line number the message points to.
- Copying code without reading it.
- Giving up. Most errors take only a minute to fix once you read the message.

👨‍👩‍👧 Parent Corner

- Ask your child to read the last line of an error aloud, then explain it in their own words.
- Never fix the bug for them straight away. Ask: What did you expect? What happened instead?
- Say that bugs are normal and share a mistake you fixed at work.
- Praise the process: Great job staying calm and testing.
- Take a break if frustration rises, and come back later.

🎯 Quick Check

1. Which line of an error message should you read first?

✅ The last line. It names the error and why.

2. What does debugging mean?

✅ Finding and fixing bugs.

3. Which error means a list position does not exist?

✅ IndexError.

4. What is try / except used for?

✅ Catching an error so the program can carry on.

5. Why add print statements while debugging?

✅ To see what the values are while the program runs.